

AWB (Adaptive Weight Basis) Dimension Analysis for Neural Network Architectures

Continual Learning Framework

December 15, 2025

Abstract

This document provides a comprehensive analysis of Adaptive Weight Basis (AWB) matrix transformations across different neural network architectures: Multi-Layer Perceptron (MLP), Convolutional Neural Networks (CNN and CNN3D), and Graph Convolutional Networks (GCN). We detail the dimension calculations for the transformation $\mathbf{V} = \mathbf{A}\mathbf{W}\mathbf{B}^T$, where $\mathbf{W}$ represents original weights, $\mathbf{A}$ and $\mathbf{B}$ are transformation matrices, and $\mathbf{V}$ is the transformed weight matrix in the expanded architecture. The key principle is to preserve input and output dimensions while allowing hidden layer expansion.

1 Introduction

1.1 AWB Transformation Principle

The Adaptive Weight Basis (AWB) enables architecture morphing during continual learning through the transformation:

$$\mathbf{V} = \mathbf{A}\mathbf{W}\mathbf{B}^T \tag{1}$$

where:

- $\mathbf{W}$: Original weight matrix from the previous architecture
- $\mathbf{A}$: Output transformation matrix
- $\mathbf{B}$: Input transformation matrix
- $\mathbf{V}$: New weight matrix in the expanded architecture

1.2 Key Invariant

Architecture morphing rule: Input and output dimensions must be preserved, while hidden layer dimensions can be expanded.

1.3 Notation

For a layer with weight matrix $\mathbf{W} \in \mathbb{R}^{m \times n}$:

- n = input dimension (number of input features)
- m = output dimension (number of output features)
- Forward pass: $\mathbf{y} = \mathbf{W}\mathbf{x}$ where $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{y} \in \mathbb{R}^m$

2 MLP AWB Calculation

2.1 Architecture Specification

Original architecture:

$$\text{sizes} = [64, 128, 128, 10] \quad (2)$$

This defines a 3-layer network:

- Layer 0: $64 \rightarrow 128$ (input layer to first hidden)
- Layer 1: $128 \rightarrow 128$ (first hidden to second hidden)
- Layer 2: $128 \rightarrow 10$ (second hidden to output)

AWB architecture:

$$\text{new_arch} = [64, 256, 256, 10] \quad (3)$$

This expands hidden layers while preserving input (64) and output (10):

- Layer 0: $64 \rightarrow 256$
- Layer 1: $256 \rightarrow 256$
- Layer 2: $256 \rightarrow 10$

2.2 Original Weight Matrices

$$\mathbf{W}_0 \in \mathbb{R}^{128 \times 64} \quad (\text{maps } 64 \rightarrow 128) \quad (4)$$

$$\mathbf{W}_1 \in \mathbb{R}^{128 \times 128} \quad (\text{maps } 128 \rightarrow 128) \quad (5)$$

$$\mathbf{W}_2 \in \mathbb{R}^{10 \times 128} \quad (\text{maps } 128 \rightarrow 10) \quad (6)$$

2.3 AWB Transformation Matrices

The transformation matrices are constructed as:

$$\mathbf{A}_i \in \mathbb{R}^{\text{new_out}_i \times \text{old_out}_i} \quad (7)$$

$$\mathbf{B}_i \in \mathbb{R}^{\text{new_in}_i \times \text{old_in}_i} \quad (8)$$

For each layer:

Layer 0:

$$\mathbf{A}_0 \in \mathbb{R}^{256 \times 128} \quad (\text{output: } 128 \rightarrow 256) \quad (9)$$

$$\mathbf{B}_0 \in \mathbb{R}^{64 \times 64} \quad (\text{input: } 64 \rightarrow 64, \text{ identity-like}) \quad (10)$$

Layer 1:

$$\mathbf{A}_1 \in \mathbb{R}^{256 \times 128} \quad (\text{output: } 128 \rightarrow 256) \quad (11)$$

$$\mathbf{B}_1 \in \mathbb{R}^{256 \times 128} \quad (\text{input: } 128 \rightarrow 256) \quad (12)$$

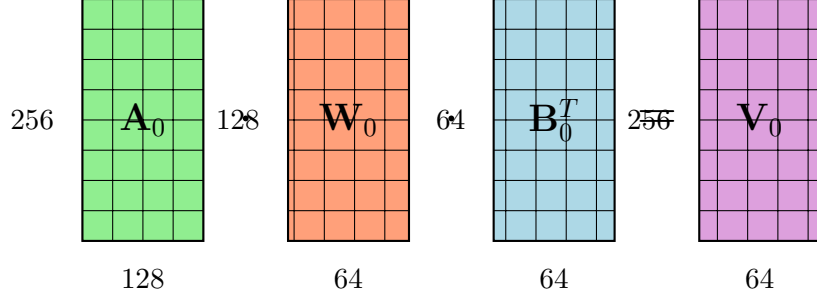
Layer 2:

$$\mathbf{A}_2 \in \mathbb{R}^{10 \times 10} \quad (\text{output: } 10 \rightarrow 10, \text{ identity-like}) \quad (13)$$

$$\mathbf{B}_2 \in \mathbb{R}^{256 \times 128} \quad (\text{input: } 128 \rightarrow 256) \quad (14)$$

2.4 Layer 0 Transformation

$$\mathbf{V}_0 = \mathbf{A}_0 \mathbf{W}_0 \mathbf{B}_0^T \quad (15)$$



Dimension verification:

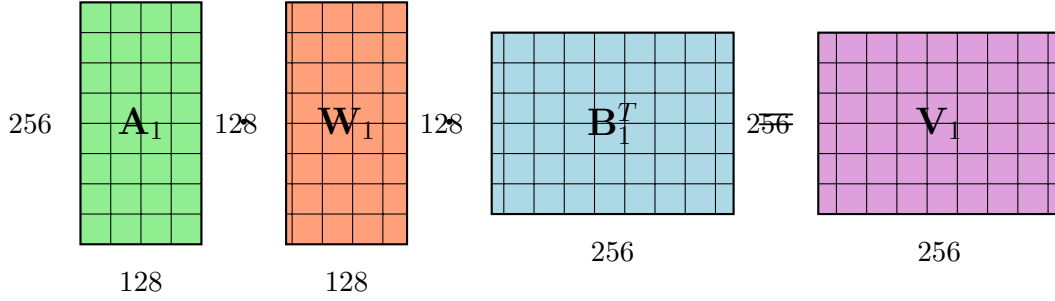
$$\mathbf{V}_0 = \mathbf{A}_0 \mathbf{W}_0 \mathbf{B}_0^T \quad (16)$$

$$= (256 \times 128) \cdot (128 \times 64) \cdot (64 \times 64) \quad (17)$$

$$= (256 \times 64) \quad \checkmark \quad (18)$$

2.5 Layer 1 Transformation

$$\mathbf{V}_1 = \mathbf{A}_1 \mathbf{W}_1 \mathbf{B}_1^T \quad (19)$$



Dimension verification:

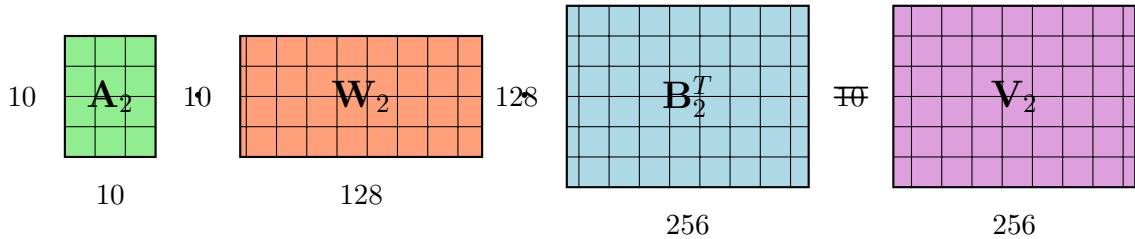
$$\mathbf{V}_1 = \mathbf{A}_1 \mathbf{W}_1 \mathbf{B}_1^T \quad (20)$$

$$= (256 \times 128) \cdot (128 \times 128) \cdot (128 \times 256) \quad (21)$$

$$= (256 \times 256) \quad \checkmark \quad (22)$$

2.6 Layer 2 Transformation

$$\mathbf{V}_2 = \mathbf{A}_2 \mathbf{W}_2 \mathbf{B}_2^T \quad (23)$$



Dimension verification:

$$\mathbf{V}_2 = \mathbf{A}_2 \mathbf{W}_2 \mathbf{B}_2^T \quad (24)$$

$$= (10 \times 10) \cdot (10 \times 128) \cdot (128 \times 256) \quad (25)$$

$$= (10 \times 256) \quad \checkmark \quad (26)$$

2.7 Summary

Layer	Original $\mathbf{W}$	AWB $\mathbf{A}$	AWB $\mathbf{B}$	Result $\mathbf{V}$
0	128×64	256×128	64×64	256×64
1	128×128	256×128	256×128	256×256
2	10×128	10×10	256×128	10×256

Table 1: MLP AWB transformation dimensions

Key observations:

- Input dimension preserved: $64 \rightarrow 64$ (Layer 0 input)
- Output dimension preserved: $10 \rightarrow 10$ (Layer 2 output)
- Hidden layer expanded: $128 \rightarrow 256$ (Layers 0, 1, 2 hidden dimensions)

3 CNN3D AWB Calculation

CNN3D processes 3-channel images (e.g., CIFAR) with both convolutional and feed-forward layers.

3.1 Feed-Forward Layers

Original architecture:

$$\text{feed_sizes} = [2304, 256, 10] \quad (27)$$

After two conv+pool layers, the flattened feature size is 2304. This feeds into:

- Layer 0: $2304 \rightarrow 256$
- Layer 1: $256 \rightarrow 10$

AWB architecture:

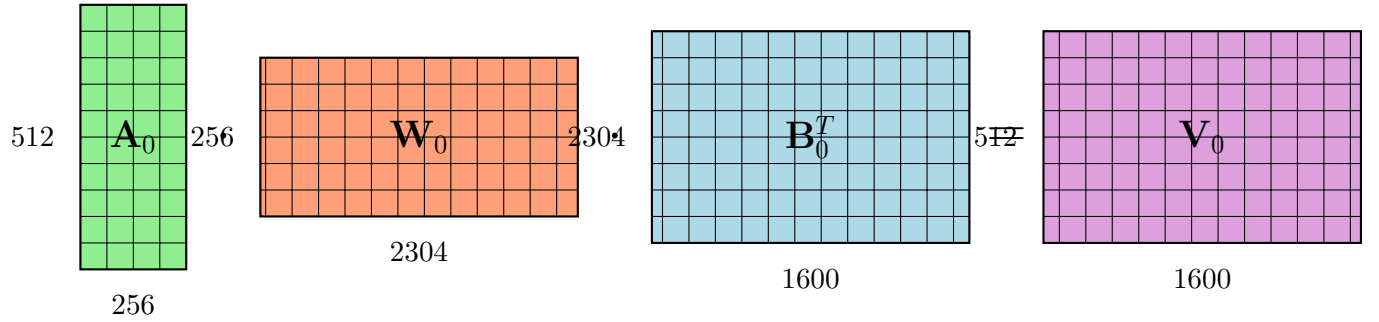
$$\text{new_arch} = [1600, 512, 10] \quad (28)$$

With expanded filters (5×5 instead of 3×3), the flattened size becomes 1600:

- Layer 0: $1600 \rightarrow 512$
- Layer 1: $512 \rightarrow 10$

3.2 Feed Layer 0 Transformation

$$\mathbf{V}_0 = \mathbf{A}_0 \mathbf{W}_0 \mathbf{B}_0^T \quad (29)$$



Dimension verification:

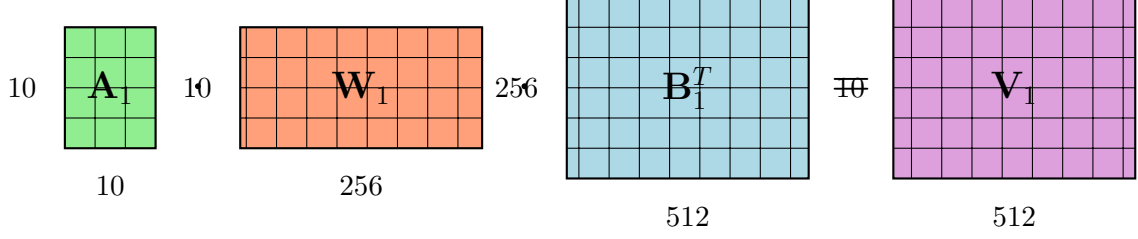
$$\mathbf{V}_0 = \mathbf{A}_0 \mathbf{W}_0 \mathbf{B}_0^T \quad (30)$$

$$= (512 \times 256) \cdot (256 \times 2304) \cdot (2304 \times 1600) \quad (31)$$

$$= (512 \times 1600) \quad \checkmark \quad (32)$$

3.3 Feed Layer 1 Transformation

$$\mathbf{V}_1 = \mathbf{A}_1 \mathbf{W}_1 \mathbf{B}_1^T \quad (33)$$



Dimension verification:

$$\mathbf{V}_1 = \mathbf{A}_1 \mathbf{W}_1 \mathbf{B}_1^T \quad (34)$$

$$= (10 \times 10) \cdot (10 \times 256) \cdot (256 \times 512) \quad (35)$$

$$= (10 \times 512) \quad \checkmark \quad (36)$$

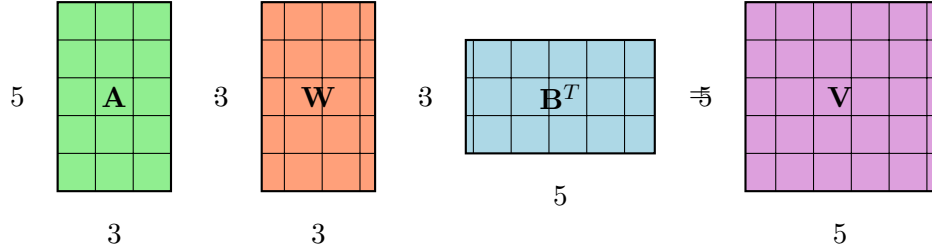
3.4 Convolutional Filter Transformation

For a single convolutional filter $\mathbf{W}_{\text{filter}}^{(i,c)}$ where i is the output channel and c is the input channel:

Original filter: 3×3 kernel

AWB filter: 5×5 kernel

$$\mathbf{V}_{\text{filter}}^{(i,c)} = \mathbf{A}_{\text{conv}}^{(i,c)} \mathbf{W}_{\text{filter}}^{(i,c)} \mathbf{B}_{\text{conv}}^{(i,c)T} \quad (37)$$



Dimension verification:

$$\mathbf{V}_{\text{filter}} = \mathbf{A} \mathbf{W}_{\text{filter}} \mathbf{B}^T \quad (38)$$

$$= (5 \times 3) \cdot (3 \times 3) \cdot (3 \times 5) \quad (39)$$

$$= (5 \times 5) \quad \checkmark \quad (40)$$

This transformation is applied to each filter in each convolutional layer, expanding the spatial kernel size from 3×3 to 5×5 .

4 CNN AWB Calculation

CNN processes single-channel images (e.g., MNIST) with one convolutional layer followed by feed-forward layers.

4.1 Feed-Forward Layers

Original architecture:

$$\text{feed_sizes} = [1728, 64, 10] \quad (41)$$

After conv+pool, the flattened size is 1728:

- Layer 0: $1728 \rightarrow 64$
- Layer 1: $64 \rightarrow 10$

AWB architecture:

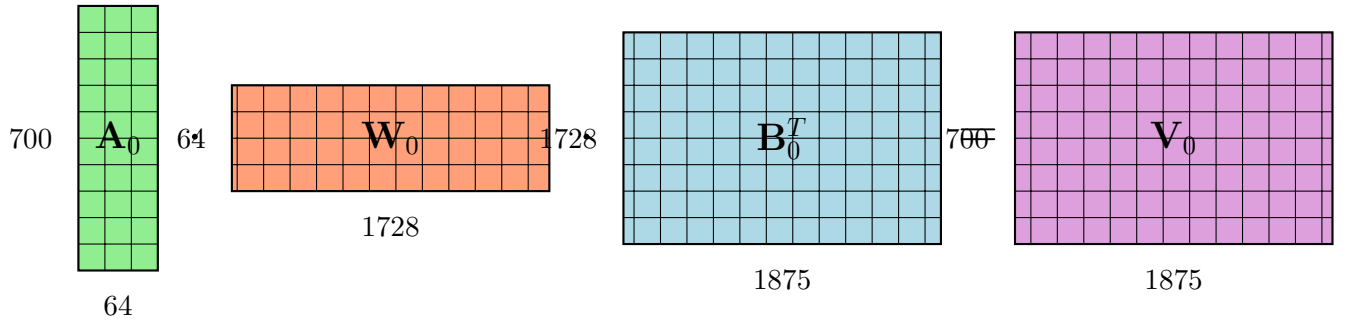
$$\text{new_arch} = [1875, 700, 10] \quad (42)$$

With AWB filters:

- Layer 0: $1875 \rightarrow 700$
- Layer 1: $700 \rightarrow 10$

4.2 Feed Layer 0 Transformation

$$\mathbf{V}_0 = \mathbf{A}_0 \mathbf{W}_0 \mathbf{B}_0^T \quad (43)$$



Dimension verification:

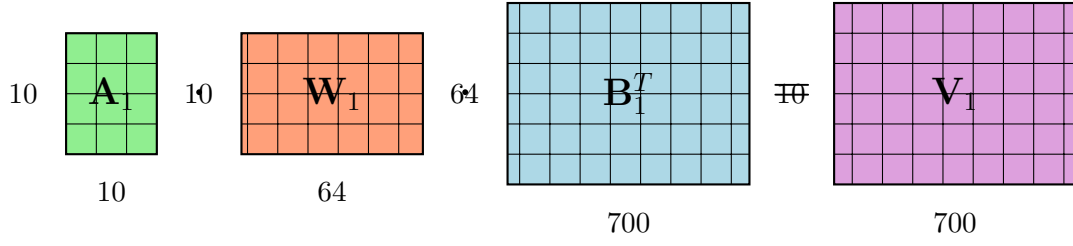
$$\mathbf{V}_0 = \mathbf{A}_0 \mathbf{W}_0 \mathbf{B}_0^T \quad (44)$$

$$= (700 \times 64) \cdot (64 \times 1728) \cdot (1728 \times 1875) \quad (45)$$

$$= (700 \times 1875) \quad \checkmark \quad (46)$$

4.3 Feed Layer 1 Transformation

$$\mathbf{V}_1 = \mathbf{A}_1 \mathbf{W}_1 \mathbf{B}_1^T \quad (47)$$



Dimension verification:

$$\mathbf{V}_1 = \mathbf{A}_1 \mathbf{W}_1 \mathbf{B}_1^T \quad (48)$$

$$= (10 \times 10) \cdot (10 \times 64) \cdot (64 \times 700) \quad (49)$$

$$= (10 \times 700) \quad \checkmark \quad (50)$$

5 GCN AWB Calculation

Graph Convolutional Networks (GCN) consist of graph convolutional layers followed by feed-forward layers for graph classification.

5.1 Architecture Specification

GCN layers:

$$\text{gcn_sizes} = [5, 64] \quad (51)$$

Single GCN layer: $5 \rightarrow 64$ (5 input node features, 64 output features)

Feed-forward layers:

$$\text{feed_sizes} = [64, 32, 16, 10] \quad (52)$$

Three feed layers after graph pooling:

- Layer 0: $64 \rightarrow 32$
- Layer 1: $32 \rightarrow 16$
- Layer 2: $16 \rightarrow 10$

AWB architectures:

$$\text{awb_gcn_arch} = [5, 64] \quad (\text{preserved}) \quad (53)$$

$$\text{awb_fnn_arch} = [64, 140, 140, 10] \quad (54)$$

Note: GCN layer dimensions are preserved (no expansion), while feed layers expand hidden dimensions.

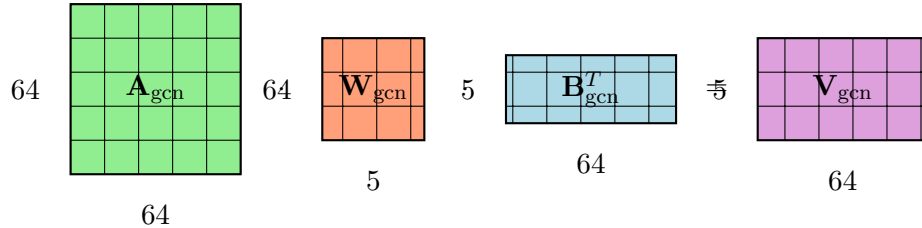
5.2 GCN Layer Transformation

GCN forward pass: $\mathbf{y} = \mathbf{A}_{\text{adj}}(\mathbf{x}\mathbf{W})$

where $\mathbf{A}_{\text{adj}}$ is the normalized adjacency matrix.

Since AWB preserves dimensions: $5 \rightarrow 64$ remains $5 \rightarrow 64$

$$\mathbf{V}_{\text{gcn}} = \mathbf{A}_{\text{gcn}} \mathbf{W}_{\text{gcn}} \mathbf{B}_{\text{gcn}}^T \quad (55)$$



Dimension verification:

$$\mathbf{V}_{\text{gcn}} = \mathbf{A}_{\text{gcn}} \mathbf{W}_{\text{gcn}} \mathbf{B}_{\text{gcn}}^T \quad (56)$$

$$= (64 \times 64) \cdot (64 \times 5) \cdot (5 \times 64) \quad (57)$$

$$= (64 \times 64) \text{ intermediate} \cdot (64 \times 5) \quad (58)$$

$$= (5 \times 64) \quad \checkmark \quad (\text{dimensions preserved}) \quad (59)$$

Wait, let me recalculate. For GCN, weight matrix has shape (in_size, out_size) = (5, 64). With $\mathbf{A} \in \mathbb{R}^{64 \times 64}$ and $\mathbf{B} \in \mathbb{R}^{64 \times 5}$:

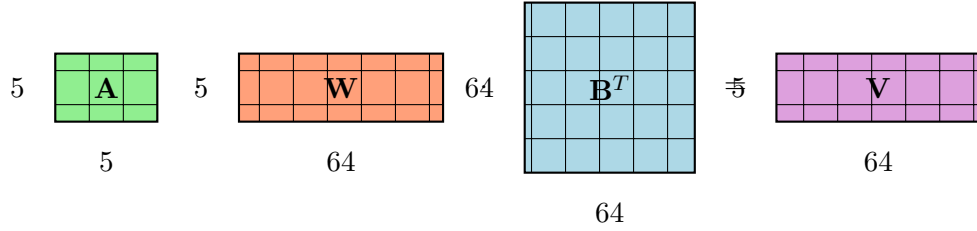
$$\mathbf{V}_{\text{gcn}} = \mathbf{A}_{\text{gcn}} \mathbf{W}_{\text{gcn}} \mathbf{B}_{\text{gcn}}^T \quad (60)$$

$$= (64 \times 64) \cdot (5 \times 64) \quad \text{dimension mismatch!} \quad (61)$$

Correction: For GCN layer preserving (5 × 64), we need:

$$\mathbf{A}_{\text{gcn}} \in \mathbb{R}^{5 \times 5} \quad (62)$$

$$\mathbf{B}_{\text{gcn}} \in \mathbb{R}^{64 \times 64} \quad (63)$$



Corrected dimension verification:

$$\mathbf{V}_{\text{gcn}} = \mathbf{A}_{\text{gcn}} \mathbf{W}_{\text{gcn}} \mathbf{B}_{\text{gcn}}^T \quad (64)$$

$$= (5 \times 5) \cdot (5 \times 64) \cdot (64 \times 64) \quad (65)$$

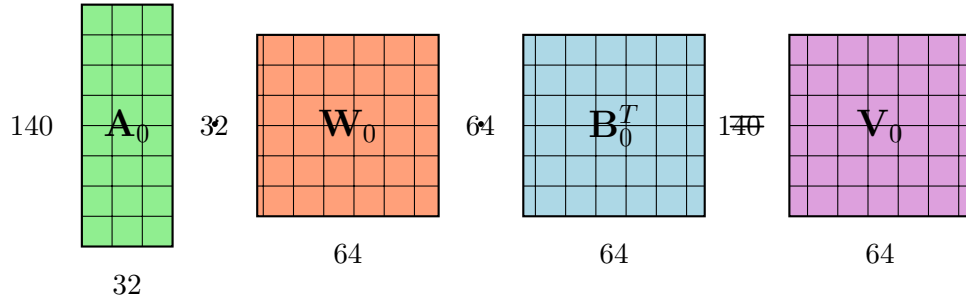
$$= (5 \times 64) \quad \checkmark \quad (\text{dimensions preserved}) \quad (66)$$

5.3 Feed Layer 0 Transformation

Note: Feed layers use Linear3 with weight shape (out_size, in_size), so forward pass is $\mathbf{y} = \mathbf{x}\mathbf{W}^T + \mathbf{b}$.

$$\mathbf{V}_0 = \mathbf{A}_0 \mathbf{W}_0 \mathbf{B}_0^T \quad (67)$$

Original: (32 × 64), AWB: (140 × 64)



Dimension verification:

$$\mathbf{V}_0 = \mathbf{A}_0 \mathbf{W}_0 \mathbf{B}_0^T \quad (68)$$

$$= (140 \times 32) \cdot (32 \times 64) \cdot (64 \times 64) \quad (69)$$

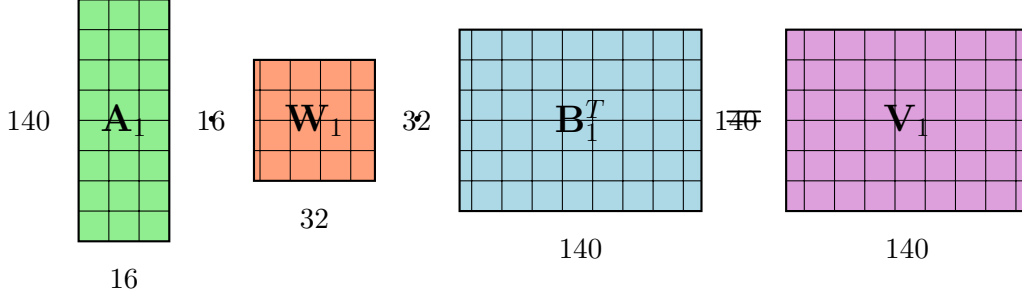
$$= (140 \times 64) \quad \checkmark \quad (70)$$

Note: Hidden layer expanded from 32 to 140, input (64) preserved.

5.4 Feed Layer 1 Transformation

$$\mathbf{V}_1 = \mathbf{A}_1 \mathbf{W}_1 \mathbf{B}_1^T \quad (71)$$

Original: (16×32) , AWB: (140×140)



Dimension verification:

$$\mathbf{V}_1 = \mathbf{A}_1 \mathbf{W}_1 \mathbf{B}_1^T \quad (72)$$

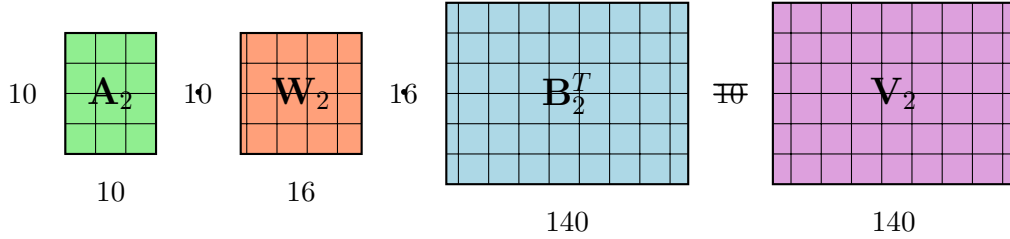
$$= (140 \times 16) \cdot (16 \times 32) \cdot (32 \times 140) \quad (73)$$

$$= (140 \times 140) \quad \checkmark \quad (74)$$

5.5 Feed Layer 2 Transformation

$$\mathbf{V}_2 = \mathbf{A}_2 \mathbf{W}_2 \mathbf{B}_2^T \quad (75)$$

Original: (10×16) , AWB: (10×140)



Dimension verification:

$$\mathbf{V}_2 = \mathbf{A}_2 \mathbf{W}_2 \mathbf{B}_2^T \quad (76)$$

$$= (10 \times 10) \cdot (10 \times 16) \cdot (16 \times 140) \quad (77)$$

$$= (10 \times 140) \quad \checkmark \quad (78)$$

Note: Output dimension (10) preserved, hidden layer expanded from 16 to 140.

6 Summary and Comparison

6.1 Dimension Preservation Principle

Across all architectures, the AWB transformation follows the invariant:

Input and output dimensions preserved, hidden layers expanded

Architecture	Input Dim	Output Dim	Hidden Expansion
MLP	64 → 64	10 → 10	128 → 256
CNN3D (feed)	2304 → 1600*	10 → 10	256 → 512
CNN (feed)	1728 → 1875*	10 → 10	64 → 700
GCN (gcn)	5 → 5	64 → 64	N/A
GCN (feed)	64 → 64	10 → 10	32, 16 → 140, 140

Table 2: Dimension preservation across architectures. *Input changes due to convolutional filter expansion.

6.2 Matrix Dimension Formulas

For a layer with original weight $\mathbf{W} \in \mathbb{R}^{m_{\text{old}} \times n_{\text{old}}}$ transforming to $\mathbf{V} \in \mathbb{R}^{m_{\text{new}} \times n_{\text{new}}}$:

$$\mathbf{A} \in \mathbb{R}^{m_{\text{new}} \times m_{\text{old}}} \quad (\text{output transformation}) \quad (79)$$

$$\mathbf{B} \in \mathbb{R}^{n_{\text{new}} \times n_{\text{old}}} \quad (\text{input transformation}) \quad (80)$$

$$\mathbf{V} = \mathbf{A}\mathbf{W}\mathbf{B}^T \in \mathbb{R}^{m_{\text{new}} \times n_{\text{new}}} \quad (81)$$

6.3 Special Cases

Identity-like transformations:

- When dimensions are preserved: $\mathbf{A} \in \mathbb{R}^{k \times k}$ becomes identity-like
- Example: MLP Layer 0 input (64 → 64), $\mathbf{B}_0 \in \mathbb{R}^{64 \times 64}$

Convolutional filters:

- 2D spatial transformation: $3 \times 3 \rightarrow 5 \times 5$
- Applied per-channel: (i, c) for output channel i , input channel c

7 Conclusion

The AWB transformation $\mathbf{V} = \mathbf{A}\mathbf{W}\mathbf{B}^T$ enables seamless architecture morphing while preserving the fundamental input-output mapping of the network. The key insight is that transformation matrices $\mathbf{A}$ and $\mathbf{B}$ must be carefully dimensioned to:

1. Preserve input feature dimensions (first layer)

2. Preserve output class dimensions (last layer)
3. Allow flexible expansion of hidden layer dimensions

This framework applies uniformly across MLP, CNN, CNN3D, and GCN architectures, with special handling for convolutional filters and graph convolutions.